P2355R2: Postfix fold expressions

Audience: EWG
S. Davis Herring <herring@lanl.gov>
March 20, 2024

History

Since r1:

- Rebased onto N4971, accounting for P2128
- Discussed additional syntax for folding over a function
- Clarified commentary

Since r0:

- Prohibited inconsistent folds like `(x[...[abc])`
- Fixed syntax in `index` example
- Used *initializer-clause* instead of redundant *assign-or-braced-init-list*

Introduction

Fold expressions work with binary operators, but not with unary operators: you can write `!!…!!x`, but there's still only one `x`. However, there are other kinds of operators to which they might apply. In particular, there are several plausible use cases for the postfix operators `[]` and `()` (function call). This paper proposes extending the *fold-expression* syntax to support these two operators. The rationale for the syntactical structure is also presented.

Syntax

The appropriate syntax may not be immediately obvious, but it can be constructed by analogy to the binary operator case. In particular, the subscripting operator is almost an ordinary binary operator already. (Infamously, the built-in operator is commutative: `1["$?"]` has the same meaning as `"$?"[1]`.) Consider how it would be supported as a binary operator `@` (to which we will not ascribe an associativity): if `x @ a` is equivalent to `x[a]`, then `x[a][b][c]` is equivalent to `((x @ a) @ b) @ c`, which is the result of the binary left fold `(x @ ... @ abc)`. Bearing in mind the implied grouping, that fold suggests the syntax

```
(x[...][abc])
```

for the case of recursive indexing.

Similarly, `(xyz[...[a]])` or `(xyz @ ... @ a)` means `x @ (y @ (z @ a))` or `x[y[z[a]]]`: a lookup with a sequence of indirections. Note the corresponding nesting in the fold-expression form. Furthermore, `(xyz[...])` is the unary right fold `(xyz @ ...)`, which is `x[y[z]]`, and the unary left fold `(...[xyz])` means `x[y][z]`; these have somewhat narrower applicability, since elements of the same pack must be usable both as containers and as indices. As *postfix-expression*s have the highest precedence, parentheses are strictly required only for the unary cases, but it is prudent to require them in all cases for consistency.

The corresponding cases for the call operator are obvious: a left fold applies each result to the next argument as a function (in a fashion similar to method chaining), while a right fold composes functions in a pack.

Of course, these operators are not strictly binary in that their right operand need not be a single expression. However, the generalization is straightforward: the right operand of a fold can be an entire (possibly empty) argument list. A unary fold must use the expansion of its operand on the left of the operator at least once, so only binary folds may be used:

```
(f(...)(abc,x))    // f(a,x)(b,x)(c,x)
(f(...)(abc,xyz))  // f(a,x)(b,y)(c,z)
(fgh(...(a,x)))    // f(g(h(a,x)))
(a[...][{ijk,0}])  // a[{i,0}][{j,0}][{k,0}]
```

Were it desired, even the cast operators would work in just the binary right fold case:

```
(static_cast<TUV>(static_cast<...>(a)))  // static_cast<T>(static_cast<U>(static_cast<V>
  (a)))
(TUV{...{a,x}})                          // T{U{V{a,x}}}
((TUV)(...)a)                            // (T)(U)(V)a
```

Placement new would similarly support just the binary left fold:

```
(new (new (x) ...) TUV)                  // new (new (new (x) T) U) V
(new (new (x) ...) T(abc,y))             // new (new (new (x) T(a,y)) T(b,y)) T(c,y)
```

These are illustrated here for completeness and to demonstrate the generality of the approach; they are certainly not proposed.

Motivation

[]

The syntactic investigation for this proposal was instigated by the discussion of multi-parameter subscripting operators. The notion of folding over `[]` was explicitly mentioned in proposals on the subject, and it can be used with types (like arrays and `std::vector`) that do not support multiple subscripting arguments:

C++23

```
// arr defines a multi-parameter
  operator[]
decltype(auto) index(auto &arr,auto
  ...ii)
{return arr[ii...];}
```

this proposal

```
// arr defines a C++20 proxy-based
  operator[]
decltype(auto) index(auto &arr,auto ...ii)
{return (arr[...][ii]);}
```

This proposal also supports the very different right fold case.

()

The convenience of fold expressions (especially when the successive subexpressions might have different types), combined with their restriction to operators, has led to common usage of workarounds involving expressing a function as an operator defined for a type that exists purely to allow a fold. This proposal allows function objects to be used instead, reducing syntactic overhead.

C++20

```
namespace detail {
  template<class F>
  struct call {
    F &&f;
    template<class T>
    decltype(auto) operator|(T &&t) const
    {return std::forward<F>(f)
  (std::forward<T>(t));}
  };
}

template<class T,class X>
decltype(auto) nest_tuple(T &&t,X &&x) {
  return std::apply
    ([&x]<class ...TT>(TT &&...tt) ->
  decltype(auto)
     {return (detail::call<TT>
  {std::forward<TT>(tt)} | ...
       | std::forward<X>(x));},
     std::forward<T>(t));
}
```

this proposal

```
template<class T,class X>
decltype(auto) nest_tuple(T &&t,X &&x) {
  return std::apply
    ([&x]<class ...TT>(TT &&...tt) ->
  decltype(auto)
     {return (std::forward<TT>(tt)(...
  (std::forward<X>(x))));},
     std::forward<T>(t));
}
```

```
namespace detail {
  template<class T>
  struct smaller {
    T &t;
    template<class U>
    auto& operator|(const smaller<U> &c)
  const {
      if constexpr(sizeof(T)<sizeof(U))
  return *this;
      else return c;
    }
  };
}

auto& smallest(const auto &...aa)
{return (detail::smaller{aa} | ...).t;}
```

or (at the cost of duplicating what could be a long signature and of requiring a great deal of inlining for efficiency)

```
auto& smallest(const auto &a) {return a;}
auto& smallest(const auto &a,const auto
  &...aa) {
  auto &b=smallest(aa...);
  if constexpr(sizeof a < sizeof b)
  return a;
  else return b;
}
```

```
auto smallest(const auto &a,const auto
  &...aa) {
  return ([&](auto &x) -> auto& {
    if constexpr(sizeof aa < sizeof x)
  return aa;
    else return x;
  }(...(a)));
}
```

Note here that the fold produces

```
ff...[0](ff...[1](...(ff...[sizeof...(aa)-1]
  (a))))
```

where `ff` is the pack of lambdas, treating `a` after `aa`; considering the lambda as a function of its capture and its actual parameter, this is

```
f(aa...[0],f(aa...[1],...f(aa...[sizeof...
  (aa)-1],a)))
```

These aren't the folds you're looking for

Of course, the usual meaning of "fold over a function" is is an expression of the form `f(a,f(b,f(c,d)))`. The above cannot directly produce such an expression, because it is a fold over `f` *itself* rather than over `()`. The `smallest` example illustrates how to assemble one anyway (given the separate `a`), and it can be generalized, but the result (treated as an opaque API) might as well be implemented with other metaprogramming:

```
template<class F,class T,class ...TT>
decltype(auto) fold(F &&f,T &&t,TT &&...tt) {
  return ([](auto &&x) -> decltype(auto)
          {return f(std::forward<TT>(tt),std::forward<decltype(x)>(x));}
          (...(std::forward<T>(t))));
}
```

There is, however, another syntax that would support the desired folds, based on extending the syntax of a fold to operators with an arity greater than 2. The observation is that expanding a fold consists of repeatedly replacing the `...` with a copy of the expression, leaving one element of the pack behind in the outer expression. When the pack has just one element left, it is substituted for the `...` instead of another copy. In the abstract, this means that

```
a : b : ... : c : wxyz
```

(for some fictitious quinary operator) expands to

```
a : b : (a : b : (a : b : w : c : x) : c : y) : c : z
```

where the placement of the pack elements is simply chosen to have them in lexical order (as is true for all existing folds).

Applying this logic to the function-call operator, interpreted not as a binary operator (applied to *a function* and *an argument list*) but as an operator of arbitrary arity applied to a function and various arguments, suggests that the syntax

```
(f(...,abcd))
```

would have the desirable expansion

```
f(f(f(a,b),c),d)
```

Note that the right-fold case would be very similar to the existing `f(abcd...)` for plain pack expansion; typically such errors would be fairly obvious. The syntax would immediately support further arguments, so long as exactly one contained an unexpanded pack. Each (and `f` itself) would be evaluated in each of its appearances, which is more expressive even if it also invites inefficiency in some cases.

One would also want binary folds (in the fold-expression sense), both for supplying an initial value (especially for empty folds, as usual) and to support packs in more than one operand (because no level of the expansion needs exactly one extra). However, it is also more syntactically difficult: repeating the n-ary operator in the usual fashion would produce

```
(f(f(x,...),abcd))
```

which would be ambiguous without the extra surrounding parentheses and requires repeating the function and verifying that it (a potentially non-trivial expression) is repeated accurately. It would be possible to supply the initial value with stripped-down syntax like

```
(f(...(x),abcd))
```

(which could directly support `fghi` as a pack as well) but then `x` appears as if it were the lone argument to some function call as well as appearing on the wrong side of `...`.

Proposal

For C++26, support unary and binary folds over the operators `[]` and `()`, with the syntax summarized below for the latter. Do not support empty unary folds for either operator for lack of an appropriate identity element. (One could argue for some sort of identity function that preserves value category for `()` (as a left identity like `void()` is for `,`), but that seems drastically inventive.)

No specific choice is proposed for the fold over `()` as an arbitrary-arity operator, pending EWG feedback. In any event, these changes do not affect the meaning of well-formed C++23 programs; the syntax is ungrammatical there.

	Example	Meaning
Binary left fold	`(f(...)({abc,0},x))`	`f({a,0},x)({b,0},x)({c,0},x)`
	`(f(...)(abc,xyz))`	`f(a,x)(b,y)(c,z)`
Binary right fold	`(fgh(...({a,0},x)))`	`f(g(h({a,0},x)))`
	`(fgh(...()))`	`f(g(h()))`
Unary left fold	`(...(abc))`	`a(b)(c)`
Unary right fold	`(abc(...))`	`a(b(c))`

Wording

Relative to N4971.

[expr.prim.fold]

Change paragraph 1:

- A fold expression performs a fold of a pack ([temp.variadic]) over a binary or postfix operator.

fold-expression:

(*cast-expression fold-operator* `...`)
(`...` *fold-operator cast-expression*)
(*cast-expression fold-operator* `...` *fold-operator cast-expression*)
(*postfix-expression* `[` `...` `]` `[` *expression-list*$_{opt}$ `]`)
(*postfix-expression* `[` `...` `[` *expression-list*$_{opt}$ `]` `]`)
(*postfix-expression* `[` `...` `]`)
(`...` `[` *assignment-expression* `]`)
(*postfix-expression* `(` `...` `)` `(` *expression-list*$_{opt}$ `)`)
(*postfix-expression* `(` `...` `(` *expression-list*$_{opt}$ `)` `)`)
(*postfix-expression* `(` `...` `)`)
(`...` `(` *assignment-expression* `)`)

[…]

Change paragraph 2:

- ~~An expression of the form~~ A *fold-expression* that begins with `(...` ~~*op* `e)` where *op* is a *fold-operator*~~ is called a *unary left fold*. ~~An expression of the form `(e` *op*~~ A *fold-expression* that ends with `...)`

~~where *op* is a *fold-operator*~~, `[...]`), or `(...)`) is called a *unary right fold*. Unary left folds and unary right folds are collectively called *unary folds*. In a unary fold, the *cast-expression*, *postfix-expression*, or *assignment-expression* shall contain an unexpanded pack ([temp.variadic]).

Change paragraph 3:

- ~~An expression of the form `(`e1 *op1* `...` *op2* e2`)` where *op1* and *op2* are *fold-operators*~~ Any other *fold-expression* is called a *binary fold*. ~~In~~If a binary fold~~, *op1* and *op2* shall be the same~~ contains two *fold-operator*~~, and either e1 shall contain an unexpanded pack or e2 shall~~s, they shall be the same. A binary fold has two operands, each an expression, an *expression-list* or nothing, or a *braced-init-list*; exactly one of them shall contain an unexpanded pack~~, but not both~~. If ~~e2~~the second contains an unexpanded pack, the expression is called a *binary left fold*; it shall not end with `))` or `])`. If ~~e1~~the first contains an unexpanded pack, the expression is called a *binary right fold*; it shall not be formed with `(...)` or `[...]`. [*Example*:

  […]

  — *end example*]

[temp.variadic]

Change bullet (5.14):

- In a *fold-expression* ([expr.prim.fold]); the pattern is the ~~*cast-expression*~~ operand that contains an unexpanded pack.

Change paragraph 13:

- The instantiation of a *fold-expression* ([expr.prim.fold]) produces:

  […]

  In each case, *op* is the *fold-operator*. If there is no *fold-operator*, *op* is a notional operator that applies the subscription operator if the *fold-expression* has a `[` or the function call operator otherwise, such that `X` *op* `Y` is `X[Y]` or `X(Y)` respectively.

  [*Note*: It is possible for `Y` to be a possibly empty expression list or a *braced-init-list*. — *end note*]

  For a binary fold, `E` is generated by instantiating the ~~*cast-expression*~~ operand that did not contain an unexpanded pack.

  […]